

Soğanlı Krep Partisi

Bohan bir parti düzenliyor. Soğanlı krebi sevdiğinden dolayı, partisine yakın dükkandan onları almaya karar verir. Dükkanda N farklı tatda soğanlı krep satılır 0'dan $N - 1$ 'e numaralandırılır. Bohan her tattan N tane soğanlı krep almaya karar verir, ve toplamda N^2 soğanlı krep alınır. Her soğanlı krep, dilimler kendi çantasına konmadan önce K özdeş dilime bölünür. Çantalar 0'dan $N^2 - 1$ 'e numaralandırılmıştır. Diyelim ki $F[i]$ i çantasındaki soğanlı krepin tadını belli etsin.

Parti boyunca, Bohan bütün misafirleri oyun oynamaya davet eder. Oyunun prosedürü şu şekildedir.

İlk olarak, Bohan N misafir seçer ve onları $0, 1, \dots, N - 1$ şeklinde numaralanmış odalara götürür öyle ki her odada kesinlikle bir misafir bulunur. Kalan misafirler oyunun sonuna kadar dışarıda kalır. Bütün N oda tam olarak aynı gözükür, bu nedenle misafirler hangi odada bulunduğunu söyleyemez.

Sonrasında, her oda için i ($0 \leq i < N$), Bohan i odasına girer ve gizlice misafire $P[i]$ tam sayısını söyler, **forbidden flavor**'u temsil eder. Ayrıca misafire hangi odada olduğunu söyleyebilir veya söylemeyebilir. Bohan $[P[0], P[1], \dots, P[N - 1]]$ dizisinin $[0, 1, \dots, N - 1]$ sayılarının bir permutasyonu olduğunu garanti eder.

Sonra, N round bulunur. i -inci roundda, Bohan sıralı düzende tek tek $i - 1$ odasına $0, 1, \dots, N^2 - 1$ çantalarını getirir. $i - 1$ odasındaki misafir j çantasını alınca, şunu öğrenir:

- $F[j]$, j çantasındaki soğanlı krepin tadı, ve
- çantada kalan dilim sayısı.

Sıradaki çantayı almadan önce, $i - 1$ odasındaki misafir şuanki çantadan kaç dilim yiyeceğine karar vermelidir. Yiyeceği dilim sayısı duruma göre şu şekilde bağlıdır:

- Eğer $P[i - 1] \neq F[j]$ ise, misafir çantadan istediği kadar dilimi silip yiyebilir.
- Eğer $P[i - 1] = F[j]$ ise, misafirin dilim yemesine izin verilmez.

Unutmayın ki, misafirler önceden $F[0], F[1], \dots, F[N^2 - 1]$ dizisini bilmemektedir.

N roundun sonrasında, dışarıdaki misafirlere çantaların sırası veriliyor. Çantaları görünce, çantalardaki soğanlı krepin tadını ve kaç dilim kaldığını öğreniyorlar. Bu bilgi ile, $P[0], P[1], \dots, P[N - 1]$ değerlerine karar vermeliler.

Misafirler oyun başlamadan iletişim kurabilir. Ayrıca N ve K değerlerini de önden biliyorlar. Amacınız öyle bir strateji kodlamak ki, dışarıdan gelen misafirler her zaman doğru şekilde $P[0], P[1], \dots, P[N - 1]$ değerlerini tahmin edebilsin.

Kodlama Detayları

Üç prosedür kodlamalısınız.

Aşağıdaki iki prosedürü $0, 1, \dots, N - 1$ odalarındaki misafirler için kodlamalısınız.

```
void init(int N, int K, int p, int r)
```

Varsayalım ki misafir i odasında olsun.

- N : soğanlı kreplerin tat sayısı.
- K : oyunun başında soğanlı krepin dilim sayısı.
- $p = P[i]$: i odasında yenmesi yasaklanan tat.
- Eğer Bohan misafire hangi odada olduklarını söylemek isterse, o zaman $r = i$. Onun dışında, $r = -1$.

```
int strategy(int b, int f, int s)
```

Varsayalım ki misafir i odasında olsun.

- b : şuanki çanta numarası.
- $f = F[b]$: b çantası için soğanlı krepin tadı.
- s : b çantasında kalan dilim sayısı.
- Bu prosedür bir negatif olmayan tam sayı x döndürmelidir, misafirin yemeye karar vereceği dilim sayısını temsil etmelidir.
 - Eğer $f \neq P[i]$ ise, $0 \leq x \leq s$ koşulu sağlanmalı.
 - Eğer $f = P[i]$ ise, $x = 0$ koşulu sağlanmalı.

Aşağıdaki prosedürü dışarıdaki misafirler için kodlamalısınız.

```
std::vector<int> guess(int N, int K, std::vector<int> F,  
                      std::vector<int> S)
```

- N : soğanlı kreplerin tat sayısı.
- K : oyunun başında soğanlı krepin dilim sayısı.
- F : N^2 uzunluğunda bir dizi $F[i]$ i çantasındaki soğanlı krepin tadını verir.
- S : N^2 uzunluğunda bir dizi $S[i]$ i çantasında N round sonunda kalan dilim sayısını gösterir.

- Bu prosedür N uzunluğunda bir dizi P döndürmeli, $P[i]$ i odasındaki misafire verilen sayı olmalıdır.

Her test durumu T bağımsız oyun içerir.

Jüri processi boyunca, her T oyun için, $N + 1$ process olacaktır. Her i öyle ki $0 \leq i < N$ için, process i , i odasındaki misafiri temsil eder. Process N dışarıdaki misafirleri temsil eder. Her i öyle ki $0 \leq i < N$ için, process $(i + 1)$ process i bitince başlar.

0'dan $N - 1$ 'e processler için:

- `init` tam olarak bir kere çağırılır.
- `strategy` `init`'den sonra N^2 kere çağırılır.
 - j -inci çağrı için, $b = j - 1$ olduğu garanti edilir.

Process N için:

- `guess` tam olarak bir kere çağırılır.

Jüri farklı oyunlardaki işlemleri birbirine karıştırabilir, ancak her bir oyun içindeki işlemlerin göreceli sırasının korunacağı garanti edilir.

Sınırlar

- $1 \leq T \leq 400$.
- $2 \leq N \leq 30$.
- Testteki bütün oyunlar için N^3 toplamı 27 000 değerini geçmez.
- $1 \leq K \leq N$
- $K \in \{1, 3, N\}$.
- $[P[0], P[1], \dots, P[N - 1]]$ şunun permutasyonudur: $[0, 1, \dots, N - 1]$.
- $0 \leq F[j] < N$ her j için öyle ki $0 \leq j < N^2$.
- her i için öyle ki $0 \leq i < N$ i tam olarak $[F[0], F[1], \dots, F[N^2 - 1]]$ içinde N kere geçer.
- Jüri **adaptif değildir**, öyle ki, $[P[0], P[1], \dots, P[N - 1]]$ ve $[F[0], F[1], \dots, F[N^2 - 1]]$ ilk `init` çağrısı yapılmadan sabittir.

Subtasklar

Subtask	Skor	Ek Kısıtlar
1	8	$K = 1, N = 2$.
2	7	$K = 1$ ve Bohan herkese hangi odada olduğunu söylemeye karar verir.
3	14	$K = 1$ ve $[F[i \cdot N], F[i \cdot N + 1], \dots, F[i \cdot N + N - 1]]$ şunun bir permutasyonudur: $[0, 1, \dots, N - 1]$ her i için öyle ki $0 \leq i < N$.
4	18	$K = N$.
5	21	$K = 3, N \geq 3$.
6	32	$K = 1$.

Örnek

$T = 1$ ve $N = 2, K = 2, P = [1, 0], F = [1, 0, 0, 1]$ testteki tek oyun için bu durumu düşününüz.

Diyelim ki $S[i]$ i çantasında kalan anlık dilim sayısını gösterebilir. Başlangıçta, $S = [2, 2, 2, 2]$.

Varsayalım ki misafirler aşağıdaki stratejiyi oyun başlamadan belirledi.

- Odalardaki misafirler için, eğer şuanki soğanlı krep yemelerine izin varsa:
 - Eğer şuanki soğanlı krep yiyebilecekleri ilkse, kalan dilimlerin hepsini yiyecekler.
 - Onun dışında, sadece bir dilim yiyecekler.
- Dışarıdaki misafirler için:
 - Eğer $S = [0, 0, 1, 1]$ ise, $P = [1, 0]$ olduğunu tahmin edecekler.
 - Onun dışında, $P = [0, 1]$ olduğunu tahmin edecekler.

Oyun şu şekilde ilerler.

İlk, jüri process 0 ile başlar 0 odasındaki misafir temsil edilir ve `init(2, 2, 1, -1)` çağrılır.

Başlangıç sonrası, jüri şu çağrılar yapar:

Procedure çağırısı	Return value
<code>strategy(0, 1, 2)</code>	0
<code>strategy(1, 0, 2)</code>	2
<code>strategy(2, 0, 2)</code>	1
<code>strategy(3, 1, 2)</code>	0

İlk ve dördüncü çağrı 0 döndürmek zorundadır çünkü $P[0] = F[0] = F[3] = 1$.

Bu process bitince (round 1 biter), $S = [2, 0, 1, 2]$.

Sonrasında, jüri misafir 1'i temsil eden process 1'i başlatır. `init(2, 2, 0, -1)` çağrılır. Başlangıç sonrası, jüri şu çağrılarını yapar:

Procedure çağırısı	Return value
<code>strategy(0, 1, 2)</code>	2
<code>strategy(1, 0, 0)</code>	0
<code>strategy(2, 0, 1)</code>	0
<code>strategy(3, 1, 2)</code>	1

Unutmayın ki ikinci ve üçüncü çağrılar 0 dönmek zorundadır çünkü $P[1] = F[1] = F[2] = 0$.

Bu process bitince (round 2 biter), $S = [0, 0, 1, 1]$.

Son olarak, jüri dışarıdaki misafirleri temsil eden process 2'yi başlatır. Jüri şu çağırısını yapar:

Procedure çağırısı	Return value
<code>guess(2, 2, [1, 0, 0, 1], [0, 0, 1, 1])</code>	[1, 0]

Dışarıdaki misafirler permutasyonu doğru tahmin etmiştir. Bundan, test doğru kabul edilir.

Unutmayın ki bu yaklaşım her zaman dışarıdaki misafirlerin permutasyonu doğru tahmin etmesini mümkün kılmaz.

Soru için attachment paketi buna ek olarak başka bir girdi daha barındırır.

Örnek Grader

Örnek grader sadece **test başına bir oyunu** destekler.

Input formatı:

```
N K
P[0] P[1] ... P[N-1]
F[0] F[1] ... F[N*N-1]
reveal
```

- `reveal` ya 0 yada 1 olur.

- Eğer `reveal` 0 ise, grader Bohan kimseye oda numarasını söylemeyecek şekilde davranır.
- Eğer `reveal` 1 ise, grader Bohan herkese oda numarasını söyleyecek şekilde davranır.

Eğer `guess`'den dönen dizi $[P[0], P[1], \dots, P[N - 1]]$ ile eşleşirse, örnek grader şunu yazdırır:

```
Accepted
```

Ek olarak, grader debuglama amacı için fonksiyon çağırısı yapıldı yazdırır.